

```

        for oBalloon in self.balloonList:
            status = oBalloon.update()
            if status == BALLOON_MISSED:
                # Balloon went off the top, remove it
                self.balloonList.remove(oBalloon)
                self.nMissed = self.nMissed + 1

❹ def getScore(self):
    return self.score

❺ def getCountPopped(self):
    return self.nPopped

❻ def getCountMissed(self):
    return self.nMissed

❼ def draw(self):
    for oBalloon in self.balloonList:
        oBalloon.draw()

```

Listing 11-6: The `BalloonMgr` class

When instantiated, the balloon manager is told the width and height of the window ❶, and it saves this information in instance variables.

The concept behind the `start()` method ❷ is important. Its purpose is to initialize any instance variables needed for one round of the game, so it's called whenever the user starts a round of the game. In this game, `start()` resets the count of popped balloons and the count of missed balloons. It then goes through a loop that creates all the `Balloon` objects (randomly chosen among three different sizes using three different classes) and stores them in a list ❸. Whenever the method creates a `Balloon` object, it passes the window and the width and height of the window. (For future expansion, each `Balloon` object is given a unique number.)

Each time through the main loop, the main code calls the `handleEvent()` method of the balloon manager ❹. Here, we check if the user has clicked on any `Balloon`. If the event detected was a `MOUSEDOWNEVENT`, the code loops through all the `Balloon` objects, asking each one if the click occurred inside that balloon. Each `Balloon` returns a Boolean indicating if it was hit and, if so, the number of points the user should get for popping it. (The code is set